

Algorithmic Pipeline of the Family Metro Map Visualizer

Implementation Documentation

September 2026

1 Introduction

This document details the exact execution pipeline of the layout algorithm implemented in `familymetromap.js` (and `familymetromap_3.js`). The system accepts an undirected tree and renders an octilinear schematic map where edges follow straight or 45°-bended paths governed by node gender parity and clearance heuristics.

2 Algorithmic Pipeline

2.1 Phase 1: Tree Verification (`verifyGraphIsTree`)

Prior to running the placement algorithm, the graph structure is validated:

1. **Non-Empty Check:** The node set V must contain at least one vertex.
2. **Degree Summation:** The sum of degrees across all vertices is accumulated to derive the total undirected edge count:

$$|E| = \frac{1}{2} \sum_{u \in V} \deg(u)$$

3. **Euler Characteristic for Trees:** The algorithm confirms the acyclic condition:

$$|E| = |V| - 1$$

4. **Reachability Check:** A Breadth-First Search (BFS) starting from the initial vertex ensures that all $|V|$ vertices reside within a single connected component.

2.2 Phase 2: Root Selection and Initialization

1. **Root Identification:** The root node r is read from the user input field (`#startNode`) or defaults to the first indexed key in the graph dictionary.
2. **Base Placement:** The root is assigned static seed coordinates:

$$\mathbf{p}_r = (350, 220)$$

3. **Gender Derivation (`getNodeGender`):** Node gender is determined via explicit attribute if present (`dataset.gender`), otherwise via character parity:

$$\text{gender}(v) = \begin{cases} \text{Female (F)}, & \text{if } \text{ASCII}(v_0) \equiv 0 \pmod{2} \\ \text{Male (M)}, & \text{if } \text{ASCII}(v_0) \equiv 1 \pmod{2} \end{cases}$$

4. **Angular Orientation Index (uK):** The base angle index $k \in \{0, \dots, 7\}$ (representing steps of $\frac{\pi}{4}$) is set to:

$$k(r) = \begin{cases} 2 & (90^\circ \text{ orthogonal}), \quad \text{if } \text{gender}(r) = \text{F} \\ 3 & (135^\circ \text{ diagonal}), \quad \text{if } \text{gender}(r) = \text{M} \end{cases}$$

5. The state tuple $(r, \text{gen} = 0)$ is enqueued into the BFS queue.

2.3 Phase 3: BFS Traversal and Generation Scaling

At each step, a vertex u with generation depth g is dequeued. For all unvisited adjacent neighbors $v \in \text{Adj}(u)$, the generation distance is scaled by the contraction factor $\alpha \in (0, 1]$:

$$d_g = 90 \cdot \alpha^g$$

2.4 Phase 4: Candidate Path Evaluation ($\text{choices} \in \{-1, +1\}$)

For each child v , the algorithm evaluates binary turn options ($\text{turn} = -1$ and $\text{turn} = +1$) based on the gender parity of (u, v) :

2.4.1 Case A: Same Gender ($\text{gender}(u) = \text{gender}(v)$)

The connection is drawn as a direct straight segment:

$$k_{\text{next}} = (uK) \pmod{8}$$

$$\mathbf{p}_v = \mathbf{p}_u + d_g \begin{bmatrix} \cos(k_{\text{next}} \cdot \frac{\pi}{4}) \\ \sin(k_{\text{next}} \cdot \frac{\pi}{4}) \end{bmatrix}$$

No intermediate bend point is produced.

2.4.2 Case B: Different Gender ($\text{gender}(u) \neq \text{gender}(v)$)

The connection introduces an intermediate 45° bend vertex. The edge is partitioned into two equal subsegments of length:

$$L = \frac{d_g}{\sqrt{2} + \sqrt{2}} \approx 0.541 \cdot d_g$$

The bend location $\mathbf{b}$ and target coordinate $\mathbf{p}_v$ are calculated as:

$$k_{\text{bend}} = (uK + \text{turn} + 8) \pmod{8}$$

$$\mathbf{b} = \mathbf{p}_u + L \begin{bmatrix} \cos(k_{\text{bend}} \cdot \frac{\pi}{4}) \\ \sin(k_{\text{bend}} \cdot \frac{\pi}{4}) \end{bmatrix}$$

$$k_{\text{final}} = (uK + 2 \cdot \text{turn} + 8) \pmod{8}$$

$$\mathbf{p}_v = \mathbf{b} + L \begin{bmatrix} \cos(k_{\text{final}} \cdot \frac{\pi}{4}) \\ \sin(k_{\text{final}} \cdot \frac{\pi}{4}) \end{bmatrix}$$

2.5 Phase 5: Collision Detection and Clearance Optimization

For each candidate trajectory, two quality measures are checked:

1. **Intersection Count:** The orientation test (ccw) evaluates geometric segment crossings against all previously committed edge lines.

2. **Distance Clearance:** The minimum Euclidean distance from $\mathbf{p}_v$ to all existing node positions is measured:

$$\delta_{\min}(\mathbf{p}_v) = \min_{w \in V_{\text{placed}}} \|\mathbf{p}_v - \mathbf{p}_w\|_2$$

The algorithm selects the candidate that minimizes total segment intersections; in the event of a tie, it selects the configuration that maximizes $\delta_{\min}(\mathbf{p}_v)$.

2.6 Phase 6: State Commitment

- Winning coordinates $\mathbf{p}_v$ and orientation k_v are stored in `coords` and `orientations`.
- If applicable, intermediate bend coordinates $\mathbf{b}$ are registered in `localBends` under key `u-v`.
- The line segments are appended to the obstacle array `linesDrawn`.
- Vertex v with generation $g + 1$ is enqueued into the BFS queue.

2.7 Phase 7: Normalization and Canvas Rendering

1. **Bounding Bounding-Box Calculation:** The global minima ($\min X, \min Y$) across all nodes and bend points are obtained.
2. **Margin Shifting:** Positive translation offsets ensure no graphical element clips past the upper-left boundary:

$$\begin{aligned}\Delta X &= \max(0, 50 - \min X) \\ \Delta Y &= \max(0, 50 - \min Y)\end{aligned}$$

3. **DOM Node Placement:** Node positions are assigned to DOM styles with center offsets, applying CSS classes `.node-male` or `.node-female`.
4. **SVG Edge Drawing:** Straight links and two-segment bended links through `window.edgeBends` are rendered into the SVG canvas.